Umbraco MCP with Copilot Enterprise agents

How agentic AI could help TPR's content managers first, and the developers who support them

Created by Alex Beesley | The Pensions Regulator, Web Platform | 9 October 2026 | Discussion draft

1. Summary

Umbraco now provides official Model Context Protocol (MCP) servers. An MCP server is a safe, controlled connector that lets an AI assistant work with an Umbraco site through the same API that powers the back office. In plain terms, it means a content manager could ask an AI assistant to find, check, tidy or draft content on our Umbraco websites, and the assistant could do it, within that person's existing permissions.

There are two sides to this. **Content managers** would use Umbraco's **Editor MCP**, which signs in as the editor themselves, so the assistant can only do what that editor is already allowed to do. **Developers** would use the **Developer MCP** alongside GitHub Copilot in VS Code, using the Copilot Enterprise licences TPR already has.

This report puts content managers first, because that is where most of the day-to-day value sits: content audits, consistency checks, accessibility checks and drafting. The developer section at the end covers setup, safety settings and a demo.

The ask: agreement to run a time-boxed pilot (four to six weeks). Developers prove the approach safely on local and test environments first, then a small group of content managers try the Editor MCP on a test site, with information governance and security sign-off before any wider use.

PART A: FOR CONTENT MANAGERS

2. What it would mean for content managers

Today, checking content across our Umbraco sites is largely manual: clicking through the tree, opening pages one by one, and keeping notes in spreadsheets. With the Editor MCP, a content manager could describe the job in plain English and let the assistant do the legwork, then review what it found or drafted before anything is published.

Task	What you could ask	Benefit
Content audits	"List every page under Guidance that has no summary, or a summary over 160 characters."	Hours of clicking replaced by a list in minutes
Finding content	"Find all pages that mention the old scheme return deadline."	Faster, more complete updates when policy or dates change
Review dates and ownership	"Which pages in this section have not been updated for over a year?"	Keeps guidance current and helps plan reviews
Consistency and house style	"Check these pages for inconsistent headings and terminology, and suggest fixes."	More consistent content across the sites
Accessibility checks	"Find images in this section with missing alt text, and empty or skipped headings."	Supports our WCAG commitments; issues flagged early
Drafting	"Create a draft page from these notes using the standard guidance page type. Do not publish."	Faster first drafts, still reviewed by a person
Tidying up	"Show me unpublished pages that still have published pages beneath them."	Fewer broken journeys and orphaned pages
Before moving or removing a page	"What links to this page, and are there redirects for it?"	Safer restructures with fewer broken links

How it keeps content managers safe

- **It acts as you.** The Editor MCP signs in with your own back office account, so it can only see and change what you already can. It cannot get round user groups or start nodes.
- **Drafts, not publishing.** The pilot would be set up so the assistant saves drafts and reports changes, and a person always reviews and publishes.
- **Everything is logged.** Changes appear in Umbraco's audit history against your name, just as if you had made them by hand.
- **Test site first.** Content managers would try it on a test environment before anyone considers live content.

What content managers would need

- Their normal Umbraco back office account, with their usual permissions.
- An AI assistant approved by TPR that can connect to MCP servers. Which assistant content managers use needs agreeing with IT and information governance, since Copilot in VS Code is a developer tool.
- A short guide and a set of ready-made requests like those in the table above, which the pilot would produce.

> **Note:** the Editor MCP is newer than the Developer MCP. Its current release status and hosting options should be confirmed with Umbraco before the content manager phase starts.

3. Governance and safety

Agentic AI that can change a CMS needs clear guard rails, particularly for a regulator.

- **People approve every change.** Nothing is published by the assistant on its own; drafts are reviewed by a person.
- **Least privilege.** Editors act with their own permissions. Developer connections use a dedicated, restricted API user per environment.
- **No production for the Developer MCP.** Umbraco's own guidance is to use it only on local or isolated test environments.
- **Read-only and dry-run first.** Both servers can be limited to reading, or set to preview changes without making them.
- **Audit trail.** Every action is recorded in Umbraco against a named person or API user.
- **Copilot policy.** An organisation owner controls the "MCP servers in Copilot" policy and can limit it to an approved list of servers.
- **Data handling.** Content read by the assistant is sent to the chosen AI model. The pilot should use non-sensitive content, and information governance should confirm the position first.

4. Proposed rollout

Phase	Who and where	Done when
1. Developer proof (2 weeks)	Web Platform developers; local and SIT; read-only and dry-run by default	Audits, checks and drafts shown working safely end to end
2. Content manager pilot (4 weeks)	A small group of content managers using the Editor MCP on a test site, drafts only	Time saved on 3 to 5 real content tasks; editors find it useful; no unintended changes
3. Sign-off	Information governance, security and IT review the pilot	Agreed position on data, assistant choice and permissions
4. Wider use	More content teams; shared guide and request library; developer patterns standardised	Part of normal practice

PART B: DEVELOPER SECTION

5. What developers get

Server	Who it is for	How it connects	Fit for TPR
CMS Developer MCP @umbraco-cms/mcp-dev @lts-17	Developers	Runs locally via npx (Node 22+); signs in as a dedicated Umbraco API user with client credentials	Use now on localhost and SIT. Never production.
CMS Editor MCP	Content managers	Best hosted; signs in as the editor's own back office user	Content manager pilot (Part A)
Forms Developer MCP	Developers	Same model as the Developer MCP	Only where Umbraco Forms is used
Custom MCP SDK	Developers	Build our own tools over our own APIs	Later, for TPR-specific services

The Developer MCP exposes most Management API operations as tools, grouped into collections such as *document*, *document-type*, *data-type*, *template*, *media*, *health*, *log-viewer*, *indexer*, *searcher*, *redirect*, *user-group* and *webhook*. Three settings matter most for safety:

- **Tool filtering:** `UMBRACO_INCLUDE_TOOL_COLLECTIONS`, plus slices such as `UMBRACO_INCLUDE_SLICES=read,list,search` or `UMBRACO_EXCLUDE_SLICES=delete`.
- **Read-only mode:** `UMBRACO_READONLY=true` removes every tool that changes anything.
- **Dry-run mode:** `UMBRACO_DRY_RUN=true` makes write tools return a preview without calling Umbraco.

Developer use cases

Area	What the agent can do
CMS and code together	Read a document type, then trace each property through Razor views, view components and services in the repo
Content modelling	Inspect document types, compositions and data types; draft new element types; find unused ones
Diagnostics	Group recent Error and Fatal log entries, run health checks, and locate the likely code path
Search	Read Examine indexer state, query searchers, rebuild an index on a test environment
Test data	Create realistic draft content on localhost or SIT for testing new components

6. Setting it up on a work laptop

These steps assume a TPR Windows laptop with VS Code and Copilot Enterprise. If IT policy blocks a step, raise it through the normal route.

Step 1: Check the prerequisites

- VS Code 1.101 or later with GitHub Copilot Chat, signed in with the TPR GitHub account.
- Node.js 22 or later: run `node -v`.
- npm can get through the corporate proxy: run `npm view @umbraco-cms/mcp-dev@lts-17 version`. A version number back means npx will work.
- The "MCP servers in Copilot" policy is enabled for our Copilot seats. It is off unless an organisation owner turns it on.

Step 2: Run a local Umbraco 17 site

Run the corporate website solution locally from Visual Studio or `dotnet run` and note the local port it starts on. A local site is the safest place for the demo.

Step 3: Create a dedicated API user

1. In the back office go to **Users**, then **Create**, then **API User**.
2. Name it clearly, for example *Copilot MCP (local)*, and add it to a user group (Administrators on localhost only; a restricted group on SIT).
3. Add **Client credentials**. The client ID must start with `umbraco-back-office-`, for example `umbraco-back-office-copilot-local`. Copy the secret straight away.

Step 4: Add the MCP server to the workspace

Paste Prompt 0 from section 7 into Copilot Agent mode, or create `.vscode/mcp.json` by hand:

```
{
  "servers": {
    "umbraco-mcp": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@umbraco-cms/mcp-dev@lts-17"],
      "env": {
        "UMBRACO_CLIENT_ID": "${input:umbraco-client-id}",
        "UMBRACO_CLIENT_SECRET": "${input:umbraco-client-secret}",
        "UMBRACO_BASE_URL": "${input:umbraco-base-url}",
        "UMBRACO_INCLUDE_TOOL_COLLECTIONS": "document,document-type,data-type,template,media,health,log-viewer",
        "UMBRACO_DRY_RUN": "true",
        "NODE_TLS_REJECT_UNAUTHORIZED": "0"
      }
    }
  },
  "inputs": [
    { "type": "promptString", "id": "umbraco-client-id",
      "description": "Umbraco API user client ID" },
    { "type": "promptString", "id": "umbraco-client-secret",
      "description": "Umbraco API user client secret", "password": true },
    { "type": "promptString", "id": "umbraco-base-url",
      "description": "Base address of the local Umbraco site" }
  ]
}
```

Note: `NODE_TLS_REJECT_UNAUTHORIZED=0` is only there to accept the self-signed local certificate; remove it for SIT. Use the `lts-17` tag, not `latest`, which targets Umbraco 18.

Step 5: Start and check the server

1. Open `.vscode/mcp.json` and click **Start** above the *umbraco-mcp* entry. VS Code prompts for the client ID, secret and site address.
2. In Copilot Chat set the mode to **Agent** and choose a model the organisation allows.
3. Click the tools icon and confirm the Umbraco tools are listed.
4. Run Prompt 1 and approve each read when VS Code asks.

Step 6: Troubleshooting

Symptom	Likely cause and fix
"MCP servers blocked by policy"	An organisation owner needs to enable "MCP servers in Copilot" and add the server to any allowlist.
Server fails to start, npx error	npm cannot get through the proxy; check npm proxy settings or ask IT.
First tool call fails with a version warning	Package tag does not match the site; use lts-17 for Umbraco 17.
401 or invalid_client	Wrong client ID or secret, or the ID is missing the umbraco-back-office- prefix.
Certificate errors on localhost	Keep NODE_TLS_REJECT_UNAUTHORIZED=0 for local only, or trust the dev certificate (dotnet dev-certs https --trust).
Agent seems confused or slow	Too many tools enabled; trim the tool collections and start a fresh chat.

7. Copilot Agent mode prompts

Paste each prompt into Copilot Chat with the mode set to Agent. They run from safest to most powerful.

Prompt 0: Set up the MCP server in this workspace

```
Set up the Umbraco Developer MCP server for this workspace.

1. Create .vscode/mcp.json (create the .vscode folder if needed). Use exactly this structure:
   - one stdio server called "umbraco-mcp"
   - command "npx", args ["-y", "@umbraco-cms/mcp-dev@lts-17"]
   - env:
       UMBRACO_CLIENT_ID      = "${input:umbraco-client-id}"
       UMBRACO_CLIENT_SECRET  = "${input:umbraco-client-secret}"
       UMBRACO_BASE_URL       = "${input:umbraco-base-url}"
       UMBRACO_INCLUDE_TOOL_COLLECTIONS = "document,document-type,data-type,template,media,health,log-viewer"
       UMBRACO_DRY_RUN        = "true"
       NODE_TLS_REJECT_UNAUTHORIZED = "0"   (add a comment that this is ONLY for a localhost self-signed
          certificate and must be removed for SIT)
   - an "inputs" array with three promptString inputs (umbraco-client-id, umbraco-client-secret with "password":
      true, umbraco-base-url).
2. Do NOT hardcode any secret, site address or client ID anywhere. Do not create a .env file.
3. Make sure .vscode/mcp.json contains no secrets, then tell me whether it is safe to commit. If the repo's
   .gitignore excludes .vscode, say so.
4. Do not run the server yourself and do not call any Umbraco tool yet. Stop and tell me to click Start above the
   server entry in mcp.json.
```

Prompt 1: Prove the connection (read only)

```
Using only the umbraco-mcp tools, and only read operations:
1. Get the server information and tell me the Umbraco version.
2. List the root content nodes with their names, document type aliases and published state.
3. List all document types, grouped by folder, with a count.
Do not create, update, publish, move or delete anything. If a tool call fails, show me the exact error and stop.
```

Prompt 2: Content quality audit (read only)

```
Using only read operations on the umbraco-mcp tools:
1. Inspect the document types and work out which property aliases hold the page title, meta description / SEO
   description and summary. Tell me the aliases you chose and why before using them.
2. Walk the content tree under the first root node, up to 3 levels deep (stop after 60 pages to keep this quick).
3. Produce a markdown table of pages where the meta description is empty, shorter than 50 characters or longer
   than 160 characters, plus any page that is unpublished but has published children.
4. Save the table to docs/ai-demo/content-audit.md in this repo. Do not change anything in Umbraco.
```

Prompt 3: Log and health triage (read only)

```
Using the umbraco-mcp health and log-viewer tools, read only:
1. Get the log level counts.
2. Pull the most recent Error and Fatal log entries (up to 50).
3. Group them by message template, show the count for each group and the most recent timestamp.
4. For the top 3 groups, search this repository for the code most likely responsible and suggest a fix, citing
   file paths and line numbers. Do not edit any files.
5. Run the Umbraco health check groups and list anything that is not a success.
```

Prompt 4: Dry-run a schema change (safe preview)

```
The MCP server is running with UMBRACO_DRY_RUN=true, so write tools only return a preview.
I want a new element type called "AI Demo Callout" with properties: heading (textstring, mandatory), body (rich
   text), link (content picker or link picker, optional).
1. First read the existing data types and pick the right ones for each property; tell me which and why.
2. Then call the create tool. Because this is dry-run, show me the preview payload that would have been sent.
3. Confirm explicitly that nothing was created in Umbraco.
```

Prompt 5: Real write on localhost only (draft, not published)

```
Before running this one: in .vscode/mcp.json set UMBRACO_DRY_RUN to "false" and restart the server. LOCALHOST
   ONLY.

Using the umbraco-mcp tools:
1. Confirm UMBRACO_BASE_URL points at localhost. If it does not, stop immediately and tell me.
2. Create a content page called "AI demo - safe to delete" under the first root node, using the simplest allowed
   child document type. Fill in title and summary with one sentence each.
3. Save it as a draft. Do NOT publish it.
4. Read it back and show me its id, document type, and that it is unpublished.
```

Prompt 6: CMS + code together (the "why this matters" demo)

```
1. Use the umbraco-mcp tools (read only) to get the full definition of the document type used by the page created
   in the previous step: its properties, aliases and editors.
2. Find in this repo the Razor view/template and any view component or controller that renders that document type.
3. Explain how each property flows from Umbraco to the rendered HTML, and list any property that exists in Umbraco
   but is never rendered, or rendered but no longer exists in Umbraco.
4. Do not change any files. Present findings as a short table plus a summary.
```

Clean up after the demo (localhost)

```
Using the umbraco-mcp tools: find the page "AI demo - safe to delete", move it to the recycle bin, then confirm it
   is in the recycle bin. Do not empty the recycle bin and do not touch anything else.
Then set UMBRACO_DRY_RUN back to "true" in .vscode/mcp.json.
```

Optional: Copilot-instructions.md snippet

```
## Umbraco MCP rules
- The umbraco-mcp server must never point at production. If UMBRACO_BASE_URL is not localhost, SIT or pre-prod,
   refuse and stop.
- Read before write: inspect document types and data types before creating or updating content or schema.
- Never publish, unpublish, delete or empty a recycle bin unless the prompt explicitly asks for that exact action.
- Prefer dry-run for schema changes and show the preview first.
- Never write secrets, client IDs or site addresses into files; use the VS Code inputs in .vscode/mcp.json.
- When a change is made, report the node id, what changed and how to undo it.
```

8. Demo running order (about 15 minutes)

Time	Show	Point to make
0 to 2 min	Part A of this report	Content managers are the main beneficiaries
2 to 4 min	mcp.json and the API user	No secrets in the repo; a dedicated, auditable identity
4 to 6 min	Prompt 1 (read only)	The assistant can see our CMS, not just our code
6 to 9 min	Prompt 2 (content audit)	The kind of audit content managers would run themselves
9 to 11 min	Prompt 4 (dry run)	Changes can be previewed without touching Umbraco
11 to 14 min	Prompt 6 (CMS and code)	Developer value: schema and code reasoned about together
14 to 15 min	Sections 3 and 4	Guard rails, then ask for the pilot

Prompt 5 (a real write) is optional; run the clean-up prompt afterwards if used. Rehearse once first, as the first npx run downloads the package.

Created by Alex Beesley

Full Stack .NET Developer, Web Platform, The Pensions Regulator